

暨南大学本科实验报告专用纸

课程名称 计算机视觉实验 指导教师 赵乘骥 成绩
实验项目名称 综合实验 实验项目编号 07
实验项目类型 实验地点 教 108 学院 国际能源学院 专业 自动化
学生姓名 曹瑞 学号 2023104666 实验时间 2026 年 5 月 8 日

一、实验目的

- 理解图像如何转换为机器学习模型可以处理的特征向量。
- 掌握训练集和测试集的划分方法与各自用途。
- 学会使用 KNN、朴素贝叶斯、逻辑回归、SVM、决策树、随机森林等传统机器学习方法完成图像分类任务。
- 掌握使用准确率、混淆矩阵、错误样本分析来评价分类模型性能。
- 对比不同分类器在相同任务上的表现差异，并分析原因。

二、数据集说明

- 数据集名称：sklearn 自带 digits 手写数字数据集
- 图像类型：8×8 灰度图像
- 总样本数量：1797 张
- 类别数量：10 类（数字 0~9）
- 特征表示方式：每张 8×8 图像按行展开为 64 维特征向量，向量元素为对应位置的像素灰度值。

三、实验方法

本次实验共使用 6 种经典机器学习分类器，满足至少 5 种方法的要求：
环境：Python 3、scikit-learn、matplotlib、pandas

表格

分类模型	简要说明
KNN	根据最近邻样本的类别投票进行分类
Naive Bayes	基于贝叶斯定理与特征条件独立假设进行概率分类
Logistic Regression	学习线性分类边界，输出类别概率进行分类
SVM	寻找具有最大间隔的最优超平面完成分类
Decision Tree	根据特征阈值逐层划分，构建决策路径进行分类
Random Forest	多棵决策树集成学习，通过投票得到最终分类结果

四、实验步骤与结果

4.1 任务 1：数据准备

- 图像总数：1797 张
- 单张图像大小：8×8
- 类别标签：[0,1,2,3,4,5,6,7,8,9]
- 已可视化并保存若干样本图像。

4.2 任务 2：数据划分

- 划分比例：训练集 75%，测试集 25%
- 训练集样本数：1347
- 测试集样本数：450
- 用途说明：
 - 训练集：用于模型学习参数、拟合数据分布。
 - 测试集：用于评估模型泛化能力，不参与训练过程。

4.3 任务 3：特征表示

1. 转换方式：8×8 二维图像按行展平，得到 64 维一维向量。
2. 转换原因：传统机器学习模型只能接收固定长度结构化向量，无法直接处理二维网格结构。
3. 原始像素特征优缺点：
 - 优点：实现简单、信息直接、无信息丢失。
 - 局限：不具备平移、旋转、尺度不变性，对噪声敏感，高维易冗余。

4.4 任务 4：模型训练与测试准确率

所有模型在相同训练集与测试集下训练与评估，结果如下：

表格

模型	测试准确率
KNN	0.9844
Naive Bayes	0.8289
Logistic Regression	0.9622
SVM	0.9911
Decision Tree	0.8244
Random Forest	0.9600

4.5 任务 5：结果比较

1. 准确率最高模型：SVM（0.9911）
2. 准确率最低模型：Decision Tree（0.8244）
3. 差异分析：
 - 不同模型表现差异明显，SVM、KNN、随机森林效果优秀；决策树、朴素贝叶斯精度偏低。

- 原因：**SVM** 在高维小样本上泛化能力强；决策树容易过拟合；朴素贝叶斯的特征独立假设与图像真实分布不符。

4.6 任务 6：错误样本分析（以 **Random Forest** 为例）

1. 混淆矩阵：已保存并可视化，可直观查看各类别混淆情况。
2. 错误样本数量：**18 / 450**
3. 最容易混淆的数字组合：
 - **8 → 1**: 3 次
 - **0 → 4**: 2 次
 - **8 → 7**: 2 次
 - **5 → 9**: 2 次
4. 错误原因：
 - 手写数字形状不规范、笔画较细。
 - 相似形状数字易混淆，如 **8** 与 **1**、**5** 与 **9**。
 - 原始像素特征无形状不变性，局部位置或灰度变化易导致误判。

五、实验总结

1. 传统机器学习处理图像的完整流程：
加载数据 → 展平为特征向量 → 划分训练 / 测试集 → 模型训练 → 评估与错误分析。
2. 各模型优缺点：
 - **SVM**：精度最高，泛化能力强，速度适中。
 - **KNN**：简单有效，但预测速度较慢。
 - 随机森林：稳定、抗过拟合，综合表现优秀。
 - 决策树：易实现但容易过拟合，精度较低。
 - 朴素贝叶斯：推理快，但假设过强，分类效果一般。
3. 原始像素特征的局限：
不具备几何不变性，对平移、旋转、尺度、噪声非常敏感，复杂场景下效果有限。

六、思考题回答

1. 为什么传统机器学习方法需要先把图像转换为特征向量？
传统机器学习模型的输入要求是固定长度的结构化特征向量，不能直接处理二维图像网格结构，必须展平才能输入模型。
2. **KNN**、**SVM**、决策树、随机森林的分类思想有什么不同？
 - **KNN**：基于距离度量，近邻样本投票。
 - **SVM**：基于间隔最大化，寻找最优分类超平面。
 - 决策树：基于特征阈值，层级划分决策路径。
 - 随机森林：基于集成学习，多棵树随机采样训练后投票。
3. 为什么单棵决策树容易过拟合？

决策树会不断分裂节点以尽可能拟合训练数据中的噪声与细节，导致树深度过大、泛化能力下降。

4. 为什么随机森林通常比单棵决策树更稳定？

随机森林通过样本随机采样、特征随机选择构建多棵独立树，再投票融合结果，能显著降低方差，抑制过拟合，更稳定。

5. 如果图像发生平移、旋转或缩放，直接使用原始像素作为特征会有什么问题？

像素位置发生变化，向量整体分布完全改变，模型会将其视为完全不同的特征，导致分类失效。

七、附录

1. Python 源代码：lab07.py

```
1.
2. import numpy as np
3. import matplotlib.pyplot as plt
4. from sklearn.datasets import load_digits
5. from sklearn.model_selection import train_test_split
6. from sklearn.neighbors import KNeighborsClassifier
7. from sklearn.naive_bayes import GaussianNB
8. from sklearn.linear_model import LogisticRegression
9. from sklearn.svm import SVC
10. from sklearn.tree import DecisionTreeClassifier
11. from sklearn.ensemble import RandomForestClassifier
12. from sklearn.metrics import accuracy_score, confusion_matrix,
    ConfusionMatrixDisplay
13. import pandas as pd
14. import os
15.
16. # 创建输出图片的目录
17. os.makedirs("results", exist_ok=True)
18.
19. # ===== 任务 1: 数据准备 =====
20. print("=" * 50)
21. print("任务 1: 数据准备")
22. digits = load_digits()
23. X = digits.images          # 形状: (1797, 8, 8)
24. y = digits.target          # 标签 0~9
25.
26. n_samples, h, w = X.shape
27. print(f"图像数量: {n_samples}")
28. print(f"图像大小: {h}x{w}")
29. print(f"类别标签: {np.unique(y)}")
30.
31. # 显示若干张样本图像
```

```

32.fig, axes = plt.subplots(2, 5, figsize=(8, 4))
33.for i, ax in enumerate(axes.flat):
34.    ax.imshow(X[i], cmap='gray')
35.    ax.set_title(f"Label: {y[i]}")
36.    ax.axis('off')
37.plt.suptitle("Sample Images from Digits Dataset")
38.plt.tight_layout()
39.plt.savefig("results/样本图像示例.png", dpi=150)
40.plt.show()
41.
42.# ===== 任务 2: 数据划分 =====
43.print("\n" + "=" * 50)
44.print("任务 2: 数据划分")
45.X_flat = digits.data # shape (1797, 64)
46.X_train, X_test, y_train, y_test = train_test_split(
47.    X_flat, y, test_size=0.25, random_state=42, stratify=y
48.)
49.print(f"训练集样本数: {X_train.shape[0]}")
50.print(f"测试集样本数: {X_test.shape[0]}")
51.print("说明: 训练集用于学习模型参数, 测试集用于评估模型泛化能力。")
52.
53.# ===== 任务 3: 特征表示 =====
54.print("\n" + "=" * 50)
55.print("任务 3: 特征表示")
56.print("每张 8x8 图像按行展开成 64 维向量 (像素灰度值)。")
57.print("传统机器学习方法通常要求输入为固定长度的特征向量, 无法直接处理二维图像结构。")
58.print("原始像素特征的优点: 简单、直接, 保留全部视觉信息; ")
59.print("局限: 缺乏平移/旋转/尺度不变性, 对噪声敏感, 维度可能较高。")
60.
61.# ===== 任务 4: 模型训练 =====
62.print("\n" + "=" * 50)
63.print("任务 4: 模型训练与评估")
64.
65.models = {
66.    "KNN": KNeighborsClassifier(),
67.    "Naive Bayes": GaussianNB(),
68.    "Logistic Regression": LogisticRegression(max_iter=10000, random_state=42),
69.    "SVM": SVC(random_state=42),
70.    "Decision Tree": DecisionTreeClassifier(random_state=42),
71.    "Random Forest": RandomForestClassifier(random_state=42)
72.}
73.
74.results = {}
75.for name, model in models.items():
76.    model.fit(X_train, y_train)
77.    y_pred = model.predict(X_test)

```

```

78.     acc = accuracy_score(y_test, y_pred)
79.     results[name] = acc
80.     print(f"{name}: 测试准确率 = {acc:.4f}")
81.
82. # ===== 任务 5: 结果比较 =====
83. print("\n" + "=" * 50)
84. print("任务 5: 准确率比较表格")
85. df_result = pd.DataFrame(list(results.items()), columns=["模型", "测试准确率"])
86. df_result["测试准确率"] = df_result["测试准确率"].apply(lambda x: f"{x:.4f}")
87. print(df_result.to_string(index=False))
88.
89. best_model_name = max(results, key=results.get)
90. worst_model_name = min(results, key=results.get)
91. print(f"\n 准确率最高: {best_model_name} ({results[best_model_name]:.4f})")
92. print(f" 准确率最低: {worst_model_name} ({results[worst_model_name]:.4f})")
93. print("不同模型间表现差异较为明显（如决策树与 SVM 之间），原因在于模型假设与复杂度不同。")
94. print("线性模型（如逻辑回归）对非线性特征拟合能力有限；树模型易过拟合；SVM 和随机森林泛化能力较好。")
95.
96. # ===== 任务 6: 错误样本分析 =====
97. print("\n" + "=" * 50)
98. print("任务 6: 错误样本分析（选择 Random Forest）")
99.
100. best_model = models["Random Forest"]
101. y_pred_best = best_model.predict(X_test)
102.
103. # 混淆矩阵
104. cm = confusion_matrix(y_test, y_pred_best)
105. disp = ConfusionMatrixDisplay(confusion_matrix=cm,
    display_labels=digits.target_names)
106. fig, ax = plt.subplots(figsize=(8, 6))
107. disp.plot(ax=ax, cmap='Blues', values_format='d')
108. plt.title("Confusion Matrix - Random Forest")
109. plt.savefig("results/混淆矩阵_随机森林.png", dpi=150)
110. plt.show()
111.
112. # 找出错误分类样本
113. errors = np.where(y_pred_best != y_test)[0]
114. print(f"错误样本数量: {len(errors)} / {len(y_test)}")
115.
116. # 分析哪些数字最易混淆
117. error_pairs = {}
118. for idx in errors:
119.     true_label = y_test[idx]
120.     pred_label = y_pred_best[idx]

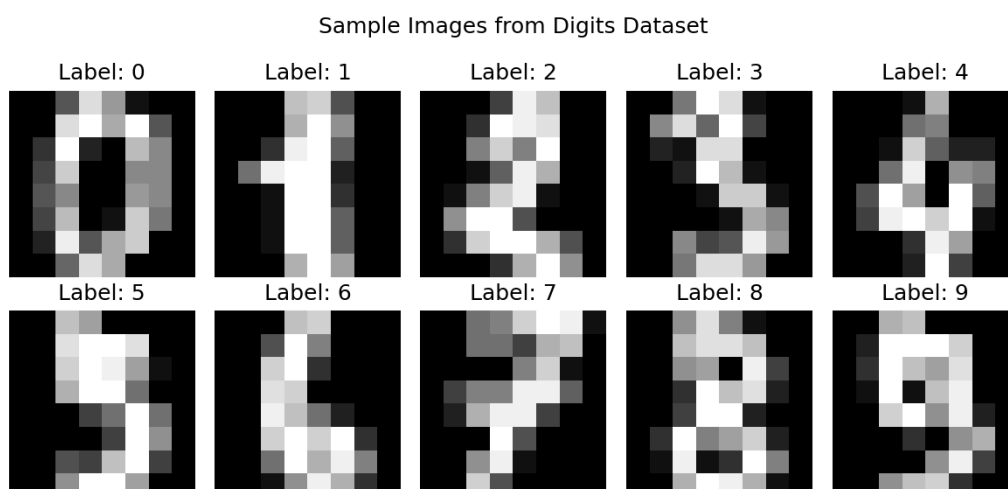
```

```

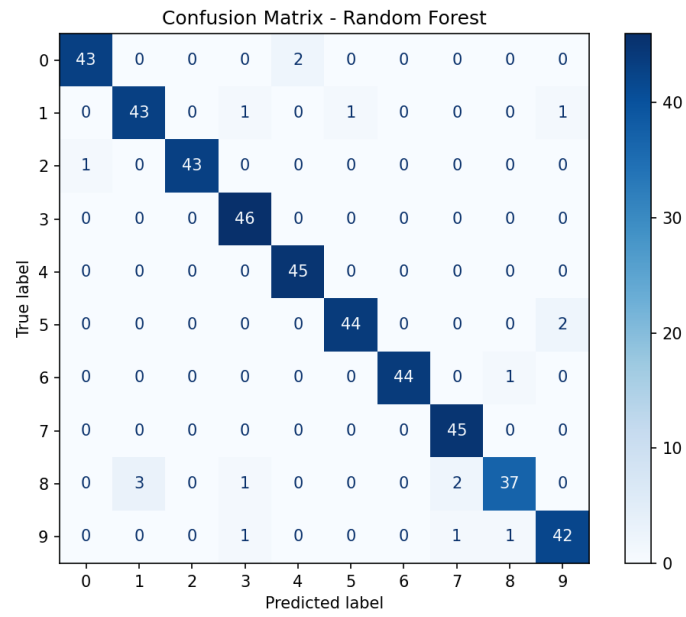
121.     if true_label != pred_label:
122.         pair = (true_label, pred_label)
123.         error_pairs[pair] = error_pairs.get(pair, 0) + 1
124.
125. # 按错误次数排序，取前 5 组
126. sorted_pairs = sorted(error_pairs.items(), key=lambda x: x[1], reverse=True)
127. print("\n 最常混淆的数字组合（真实 → 预测）及次数:")
128. for (true, pred), count in sorted_pairs[:5]:
129.     print(f" {true} → {pred}: {count} 次")
130.
131. # 展示若干错误分类样本
132. n_show = min(6, len(errors))
133. fig, axes = plt.subplots(1, n_show, figsize=(12, 3))
134. for i, ax in enumerate(axes.flat):
135.     idx = errors[i]
136.     ax.imshow(X_test[idx].reshape(8, 8), cmap='gray')
137.     ax.set_title(f"True:{y_test[idx]}\nPred:{y_pred_best[idx]}")
138.     ax.axis('off')
139. plt.suptitle("Misclassified Samples (Random Forest)")
140. plt.tight_layout()
141. plt.savefig("results/错误分类样本_随机森林.png", dpi=150)
142. plt.show()
143.
144. print("\n 分析： 错误样本通常笔画细、形状不规范或与其它数字相似，比如 8 与 3、5 与 3
    等。")
145. print("像素级别特征缺乏形状不变性，模型容易受局部噪声或书写风格影响。")
146.
147. print("\n" + "=" * 50)
148. print("实验完成，所有结果图片已保存至 results 文件夹。")

```

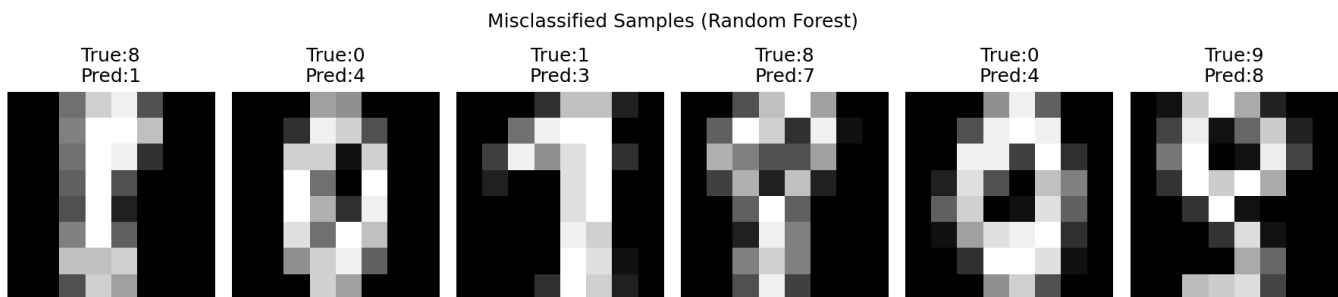
2. 结果图片:



results / 样本图像示例.png



results / 混淆矩阵_随机森林.png



results / 错误分类样本_随机森林.png